C5 - Agent Tools: Reading Summary, Server Critique, and Reflection

Course: EECE 503P/798S: Agentic Systems
Assignment: Building and Demonstrating an MCP Toolkit with FastMCP and the OpenAI Agents SDK

Part 1: Reading Summary and Existing MCP Server Critique

In "*What Is MCP, and Why Is Everyone – Suddenly! – Talking About It?*", Ksenia Se analyzes how Anthropic's Model Context Protocol (MCP) resolves the $M \times N$ integration bottleneck connecting AI models to external systems. Instead of writing bespoke connectors between M AI applications and N services, MCP establishes a bidirectional JSON-RPC 2.0 standard across three architectural roles. First, the **MCP Host** is the user-facing AI application (such as Claude Desktop, Cursor, or an OpenAI Agents SDK runner) that manages the LLM, security boundaries, and user consent. Second, the **MCP Client** lives inside the Host, maintaining an isolated, stateful one-to-one session with a paired server to handle capability discovery and message routing. Third, the **MCP Server** is a standalone process exposing **Tools** (model-invoked functions), **Resources** (application-read context), and **Prompts** (reusable templates) over local `stdio` or remote HTTP/SSE transports without exposing raw credentials to the model. Se also notes a practical trade-off: while dynamic tool discovery accelerates integration, exposing bloated or excessive tools degrades LLM routing accuracy and inflates token usage.

To see these principles applied, I explored the official **Fetch MCP Server (`mcp-server-fetch`)**, which retrieves web pages for LLM consumption. It exposes a single tool, `fetch`, governed by a Pydantic schema with four typed parameters: `url` (required `AnyUrl`), `max_length` (integer, default `5000`), `start_index` (integer, default `0`), and `raw` (boolean, default `false`). Critiqued against class tool design principles, `fetch` demonstrates a **well-defined interface** and **predictable behavior**: constraints and defaults are explicit, `robots.txt` rules are verified before fetching, chunked pagination via `start_index` prevents context overflow, and HTTP or permission failures return actionable `McpError` diagnostics rather than silent empty strings. Regarding **isolated, single-purpose functionality**, `fetch` is strictly read-only (`readOnlyHint: true`, `idempotentHint: true`), avoiding the hallucination risks and ambiguous parameter combinations inherent to large multi-purpose "Swiss-army-knife" tools. Although toggling HTML-to-Markdown extraction via `raw` slightly bundles network retrieval with content parsing, this deliberate trade-off spares the agent from passing verbose raw HTML across a second tool call.

(Word count: 323 words)

Part 2 & 3: Reflection on Tool Boundaries and Toolkit Design

Drawing clean boundaries across our **Weather & Travel Planning Assistant** toolkit required enforcing the "*no hidden 'and'*" rule—separating state-free observation, state-mutating

persistence, and workflow composition, reinforced at the protocol level with MCP tool annotations and typed Pydantic `outputSchema` models. We isolated `get_weather_forecast` as a pure asynchronous **Data tool** (`readOnlyHint=True`) whose sole job is querying the Open-Meteo Geocoding and Forecast REST APIs (with an offline fallback catalog) without touching local files. Conversely, `save_travel_itinerary` is an atomic **Action tool** (`readOnlyHint=False`) that only validates and appends a structured packing record to `data/saved_trips.json`, remaining completely agnostic to meteorological logic. Finally, `plan_weather_smart_trip` serves as our **Orchestration tool**, awaiting `get_weather_forecast`, applying domain thresholds on temperature, precipitation probability, and wind gusts to synthesize tailored gear and advisories, and awaiting `save_travel_itinerary`. Encapsulating this multi-step pipeline inside a dedicated Orchestration tool saved an extra LLM round-trip and guaranteed deterministic safety advisories while preserving the atomic Data and Action tools for standalone queries.

Three tools felt like the ideal baseline for illustrating the Data–Action–Orchestration triad: the OpenAI Agents SDK (`gpt-4o-mini`) routed read-only forecast questions, user-supplied checklists, and end-to-end trip requests with zero ambiguity. However, for a production travel assistant, three tools quickly become a lower bound because users inevitably need full lifecycle management over saved plans. Because `save_travel_itinerary` only creates new records, natural next steps would be introducing single-purpose CRUD Action tools—such as `update_packing_item_status` (to check off packed gear) and `delete_travel_itinerary`—alongside a secondary Data tool like `search_points_of_interest` so the Orchestration tool can pair weather forecasts with indoor or outdoor daily sightseeing schedules.